

Programmieren - Exkurse

Matthias Berg-Neels

zum Skript: <https://matthiasbergneels.github.io/md-scripts/>



[Download Skript](#)

Inhalt

- Naming conventions
- Implizite Typisierung mittels var
- Unit Tests
- Build & Deliver
- Innere Klassen
- Entwurfsmuster (Design Patterns)
- (Die Evolution von Java)
- (Optionals)
- (Programming Principals)

Naming conventions

Naming conventions sind, in (großen) Projekten / Firmen, Bestandteil der "Code Style Guidelines". Firmen bzw. Community basiert gibt es unterschiedliche Code Style Guidelines nach denen man sich richten kann. Die folgenden Konventionen basieren auf den [Google Style Guidelines für Java](#).

Allgemeine Regeln für Bezeichner und Namen (1/2)

Regeln

- erlaubte Zeichen
 - Buchstaben (Case sensitive): a, b, c, ..., x, y, z, A, B, ..., Y, Z
 - Landespezifische Zeichen sind erlaubt (z.B. ä, ü, ö) - sollten aber vermieden werden! (Stichwort: Kompatibilität zwischen Rechnern - betrifft NUR Quellcode, nicht den Bytecode!)
 - Unterstrich: _
 - Dollarzeichen: \$
 - Zahlen: 0, 1, 2, ..., 9
- nicht erlaubte Zeichensatz
 - Sonderzeichen

Allgemeine Regeln für Bezeichner und Namen (2/2)

weitere Regeln

- keine Leerzeichen
- dürfen nicht mit Zahlen beginnen
- dürfen nicht gleich mit reservierten Schlüsselwörtern sein

Empfehlungen

- sprechende Namen -> man kann beim Lesen verstehen was eine Variable, Klasse, Methode für eine Aufgabe hat.
- **Merksatz:** Wenn man einen Namen lesen kann ohne "grammatikalische" Bauchschmerzen zu bekommen, ist es die richtige Richtung.
- **Anmerkung:** Namen sollten immer die tatsächliche Funktion einer Entität widerspiegeln, daher stehen sie in hoher Abhängigkeit zum Quellcode. Eine Variable mit dem Namen "WindowCount" (vom Typ Boolean), eine Methode "saveToDatabase" (die nichts Speichert) oder eine Klasse "Student" (die Funktionen einer Vorlesung enthält) sollten namentlich noch einmal überdacht werden, auch wenn diese sich "rein vom Namen" korrekt lesen.

Packagenamen

Guideline

- klein geschrieben

```
de.mbn.myapp.lecture  
lecture.objectorientation.trainstation
```

Klassennamen

Guideline

- beginnen mit einem Großbuchstaben
- UpperCamelCase - beginnen mit Großbuchstaben und jedes neue Wort beginnt mit einem Großbuchstaben
- Zusatz: Der Dateiname muss sich nach dem Namen der Hauptklasse (first level) in der Datei richten

```
Car  
Student  
TrainDriver
```

Variablen / Attribute

Guideline

- lowerCamelCase - beginnen mit einem Kleinbuchstaben und jedes neue Wort beginnt mit einem Großbuchstaben

```
familyName  
children  
studentId
```


Konstanten

Guideline

- UPPER_CASE - werden vollständig mit Großbuchstaben geschrieben, neue Wörter werden durch Unterstrich getrennt

```
ALLOWED_COLOR_RED  
MEANING_OF_LIFE
```

Methoden

Guideline

- lowerCamelCase - beginnen mit einem Kleinbuchstaben und jedes neue Wort beginnt mit einem Großbuchstaben
- beginnen mit einem Verb (bzw enthalten mindestens ein Verb)
 - eine Methode "spiegelt" eine Tätigkeit, Geschehen, Vorgang wieder --> es passiert etwas

```
accelerate();  
persistData();
```

Getter- / Setter

- Attributname wird mit get bzw. set vorangestellt in lowerCamelCase

```
setFamilyName();  
getFamilyName();
```

Spezialfall: Boolean Attribute / Getter-Methoden

- sprechende Definition von Boolean-Attribute
 - z.B. enabled, isTired (VS tired), hasFlatRoof (VS flatRoof), canFly (VS fly), ...
- Boolean Getter-Methoden werden nicht mit get, sondern mit dem passenden Verb (is, has, can) gebildet
 - isEnabled(), isTired(), hasFlatRoof(), canFly()
- anhängig vom Attributnamen können in diesem Fall die Setter-Namen doch komisch wirken
 - setEnabled, setIsTired (VS setTired), setHasFlatRoof (VS setFlatRoof), setCanFly (VS setFly)

Ein Beispiel aus dem echten Leben (/ produktiven Code)

```
package com.sap.iot.rules.ruleprocessorstream.cache.repository;

import // ...

public class ThingModelBasedDataObjectRepository extends HashOperationsRepository<ThingModelBasedDataObject> {

    private static final String KEY_PREFIX = "do_by_rsid_tt_ps";

    private static final String NAME = "name";
    private static final String TYPE = "type";
    private static final String IS_RESULT = "isResult";
    private static final String THING_TYPE = "thingType";
    private static final String PROPERTY_SET = "propertySet";
    private static final String PROPERTY_SET_TYPE = "propertySetType";
    private static final String SENSITIVITY_LEVEL = "sensitivityLevel";
    private static final String LIST_SIZE_SUFFIX = "size";
    private static final String USED_ATTRIBUTES_PREFIX = "usedAttributes.";
    private static final String WILDCARD = "*";

    public ThingModelBasedDataObjectRepository(@Qualifier("ruleCache") RedisTemplate<String, String> redisTemplate, TenantContext tenantContext) {
        // ...
    }

    @Override
    public String getUniqueCachingKeyPrefix() {
        // ...
    }

    @Override
    public void save(@NotBlank String ermRuleServiceId, @Valid ThingModelBasedDataObject dataObject) {
        // ...
    }

    public Optional<ThingModelBasedDataObject> find(@NotBlank String ermRuleServiceId, String thingType, String propertySet) {
        // ...
    }

    public List<ThingModelBasedDataObject> findAll(@NotBlank String ermRuleServiceId, String thingType) {
        // ...
    }

    public Optional<ThingModelBasedDataObject> find(@NotBlank String ermRuleServiceId, String propertySetType) {
        // ...
    }

    public Boolean delete(@NotBlank String ermRuleServiceId, String thingType, String propertySet, String propertySetType) {
        // ...
    }
}
```

implizite Typisierung mittels `var`

Ermittlung des Datentyps einer Variable ohne spezifische Angabe des Typs mittels der Initialisierung - Schlüsselwort `var` (seit Java 10) `some evil stuff`

Verwendung

Voraussetzung

- Deklaration mit var UND sofortiger Initialisierung der Variable.

Beispiele

```
var numberA = 10;
    // numberA is declared as Integer variable
var numberB = 42.0;
    // numberB is declared as Double variable
var textA = "Welcome";
    // textA is declared as String variable
var myAnimal = new Dog(...);
    // myAnimal is declared as Dog variable

var test;
    // Compiler error!

int numberC = 100;

var numberD = numberC;
    // numberD is declared as Integer variable
```

Besser lesbarer (kürzerer) Code durch Vermeidung von Redundanzen

```
// vorher:
ThingModelBasedDataObjectRepository myThingModelRepo =
    new ThingModelBasedDataObjectRepository(myCacheTemplate, customerTenant);

// neu:
var myThingModelRepo = new ThingModelBasedDataObjectRepository(myCacheTemplate, customerTenant);
```

ABER...

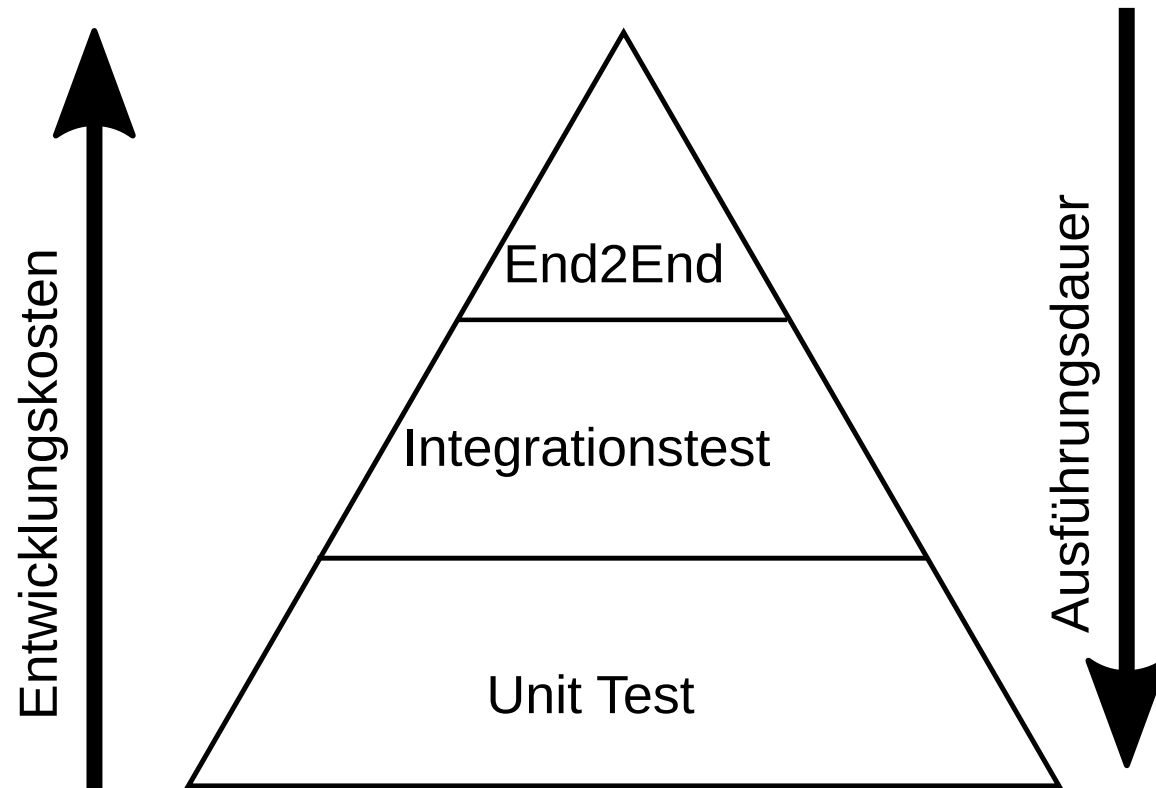
Falsch eingesetzt, wird der Code unverständlicher / komplizierter zu lesen:

```
var somethingOne = (farm.hasAnimal()) ? new Dog(...) : "No animal";  
var somethingTwo = ("Result is " + (numberA + numberB * 50.1)).length() / (double)10;  
var somethingThree = Something.returnSomething();  
// ...
```

Unit Testing

Unit Testing in Java mit JUnit5.

Einordnung von Unit Tests



JUnit5 - Basis Annotationen

Annotation	Beschreibung
@Test	kennzeichnet eine Methode als Test
@Tag(" <tag> ")	definiert einen Tag zur Filterung von Tests
@BeforeEach	kennzeichnet eine Methode die vor jedem Test läuft (JUnit4 --> @Before)
@AfterEach	kennzeichnet eine Methode die nach jedem Test läuft (JUnit4 --> @After)
@BeforeAll	kennzeichnet eine Methode die einmal vor allen Tests läuft (JUnit4 --> @BeforeClass)
@AfterAll	kennzeichnet eine Methode die einmal nach allen Tests läuft (JUnit4 --> @AfterClass)

Assertion

- Zusicherung / Sicherstellung / Assertion (lat. Aussage / Behauptung)
 - Definition einer Erwartungshaltung zum Vergleich gegen den tatsächlichen Zustand
- JUnit Tests:
 - Klasse: `Assertions` (`org.junit.jupiter.api.Assertions`)
 - statische Methoden zur Definition eines erwartenden Ergebnisses (`expected`) zum Vergleich mit dem tatsächlichen Ergebnis (`actual`)
 - überladene Methoden mit zusätzlichem Parameter `Message` für eigene Meldungen
 - automatische Validierung der "Behauptung" durch das JUnit Test-Framework
 - beliebt als statischer Import zur direkten Nutzung der Methoden:

```
import static org.junit.jupiter.api.Assertions.*;
```

- Beispiele:

```
Assertions.assertEquals(<expected>, <actual>[, <Message>]);  
Assertions.assertNotEquals(<expected>, <actual>[, <Message>]);  
Assertions.assertTrue(<actual>[, <Message>]);  
Assertions.assertFalse(<actual>[, <Message>]);  
Assertions.assertTimeout(<expected Duration>, <Executable>[, <Message>]);  
Assertions.assertThrows(<expected Exception-Class>, <Executable>[, <Message>]);
```

JUnit5 - neue Annotationen

Annotation	Beschreibung
<code>@DisplayName("descriptive name")</code>	definiert den Anzeigename für den jeweiligen Test / die Testklasse
<code>@Nested</code>	markiert eine innere (geschachtelte) Testklasse -> Strukturierung
<code>@RepeatedTest(count)</code>	sich wiederholender Testfall
<code>@ParameterizedTest</code>	Testfall Parametrisierung -> siehe nächste Slide

@ParameterizedTest

- Separierung von Test-Code und Testfall
- verschiedene Quellen für Testfälle
 - @ValueSource
 - @EmptySource / @NullSource / @NullAndEmptySource
 - @EnumSource
 - @CsvSource / @CsvFileSource
 - @MethodSource

JUnit5 - Nützliches

Testen von Ausnahmen

```
Exception assertThrows(ExceptionClass, Executable)
```

Testen von mehrer Annotationen auf einmal

```
void assertAll(Executable ...);
```

Testen der Laufzeit

```
void assertTimeout(Duration, Executable);
```

Beispiel: Einfache Test-Klasse

```
import excercises.exkurs.junit.Calculator;
import org.junit.jupiter.api.*;

class CalculatorTest {

    Calculator myCalculator;
    double result = 0;

    @BeforeEach
    void setUp() {
        myCalculator = new Calculator();
        result = 0;
    }

    @Test
    @DisplayName("adding two numbers")
    void add() {
        result = myCalculator.add(5.0, 10.0);
        Assertions.assertEquals(15.0, result);
    }
}
```

F.I.R.S.T. Principal

- **Fast:** Die Testausführung soll schnell sein, damit man sie möglichst oft ausführen kann. Je öfter man die Tests ausführt, desto schneller bemerkt man Fehler und desto einfacher ist es, diese zu beheben.
- **Independent:** Unit-Tests sind unabhängig voneinander, damit man sie in beliebiger Reihenfolge, parallel oder einzeln ausführen kann.
- **Repeatable:** Führt man einen Unit-Test mehrfach aus, muss er immer das gleiche Ergebnis liefern.
- **Self-Validating:** Ein Unit-Test soll entweder fehlschlagen oder gut gehen. Diese Entscheidung muss der Test treffen und als Ergebnis liefern. Es dürfen keine manuellen Prüfungen nötig sein.
- **Timely:** Man soll Unit-Tests vor der Entwicklung des Produktivcodes schreiben.

Build & Deliver

Bauen & Ausliefern

Was sind JAR-Dateien?

Eine **JAR-Datei** (Java Archive) ist ein komprimiertes Archiv, das Java-Klassen, Ressourcen und Metadaten zusammenfasst. Es wird häufig verwendet, um Java-Anwendungen zu verpacken und bereitzustellen. Eine JAR-Datei basiert auf dem **ZIP-Dateiformat** und wird von der Java-Plattform unterstützt.

Zweck von JAR-Dateien

- **Bereitstellung von Anwendungen:**
 - JAR-Dateien fassen alle notwendigen Dateien zusammen, um Java-Anwendungen einfach zu verteilen und auszuführen.
- **Wiederverwendbare Bibliotheken:**
 - Entwickler können Java-Bibliotheken als JAR-Dateien bereitstellen, die andere Projekte einbinden können.
- **Kompaktheit:**
 - Durch Kompression sparen JAR-Dateien Speicherplatz und reduzieren die Anzahl der Dateien, die verwaltet werden müssen.

text

Aufbau einer JAR-Datei

Eine JAR-Datei enthält:

1. **Java-Klassen:** Kompilierte `.class`-Dateien.
2. **Ressourcen:** Dateien wie Bilder, Konfigurationsdateien oder Texte.
3. **Manifest-Datei:** Eine spezielle Datei (`META-INF/MANIFEST.MF`), die Metadaten über die JAR-Datei enthält. Sie kann unter anderem die Hauptklasse definieren, die beim Start ausgeführt werden soll.

Erstellen einer JAR-Datei (manuell)

Schritt 1: Kompilieren der Quellcode-Dateien

```
javac -d out src/MyApp.java src/Helper.java
```

Schritt 2: Manifest-Datei erstellen (MANIFEST.MF)

```
Manifest-Version: 1.0  
Main-Class: MyApp
```

Schritt 3: JAR-Datei erzeugen

```
jar cfm MyApp.jar MANIFEST.MF -C out .
```

Schritt 4: JAR-Datei ausführen

```
java -jar MyApp.jar
```

*Sobald ein Projekt wächst (viele Klassen, externe Bibliotheken), wird dieser manuelle Prozess schnell unhandlich → **Build-Tools** schaffen Abhilfe!*

IntelliJ IDEA als Build-Umgebung

*IntelliJ IDEA ist eine **Integrated Development Environment (IDE)** von JetBrains. Sie übernimmt intern den Build-Prozess und verbirgt die Komplexität von `javac` und `jar` hinter einer grafischen Oberfläche.*

Was IntelliJ automatisch erledigt:

- Kompiliert den Code im Hintergrund bei jeder Änderung (**Auto-Build**)
- Verwaltet den Classpath (welche Klassen / Bibliotheken verfügbar sind)
- Zeigt Fehler sofort im Editor an – noch vor dem eigentlichen Build
- Führt Programme mit einem Klick oder Tastenkürzel aus (`Shift + F10`)

IntelliJ – Projektstruktur

IntelliJ verwaltet Projekte über eine eigene Konfiguration (`.idea/`-Ordner + `.iml`-Dateien):

```
mein-projekt/  
├── .idea/  
│   ├── IntelliJ project configuration (not in Git!)  
│   ├── workspace.xml  
│   └── modules.xml  
├── mein-projekt.iml      ← Module configuration (source folders, dependencies)  
└── src/  
    ├── de/mbn/myapp/  
    └── MyApp.java
```

Wichtige Einstellungen in IntelliJ:

- **File** → **Project Structure** (Strg+Alt+Shift+S): SDK, Quellordner, Ausgabeordner
- **Source Root**: Ordner, der als Ausgangspunkt für Packages gilt (blau markiert)
- **Output path**: Wohin `.class`-Dateien kompiliert werden (Standard: `out/`)

IntelliJ – Build & Run

Bauen:

Aktion	Menü	Tastenkürzel
Projekt bauen	Build → Build Project	Strg + F9
Einzelne Datei bauen	Build → Recompile	Strg + Shift + F9
Artefakt (JAR) erzeugen	Build → Build Artifacts	-

Ausführen:

Aktion	Menü	Tastenkürzel
Programm starten	Run → Run	Shift + F10
Debugger starten	Run → Debug	Shift + F9

IntelliJ kompiliert automatisch, bevor ein Programm ausgeführt wird!

IntelliJ – JAR-Datei erzeugen

IntelliJ kann JAR-Dateien über sogenannte **Artefakte** erzeugen:

1. File → Project Structure → Artifacts → + → JAR → From modules with dependencies
2. Hauptklasse (Main Class) auswählen
3. Build → Build Artifacts → Build
4. JAR liegt im konfigurierten Ausgabeordner (z.B. out/artifacts/)

```
out/artifacts/mein-projekt_jar/  
└─ mein-projekt.jar
```

5. Ausführen:

```
java -jar out/artifacts/mein-projekt_jar/mein-projekt.jar
```

IntelliJ – Grenzen & Einordnung

IntelliJ eignet sich gut für:

- Einzelne Entwickler / kleine Projekte
- Schnelles Ausprobieren und Lernen
- Einfache Projekte ohne viele externe Bibliotheken

Nachteile gegenüber dedizierten Build-Tools (Maven, Gradle):

- Build ist **nicht reproduzierbar** ohne IntelliJ (andere Entwickler, Server, CI/CD)
- Kein standardisiertes **Dependency Management** (Bibliotheken müssen manuell als JAR heruntergeladen werden)
- **Keine Kommandozeilen-Automatisierung** möglich
- Projektkonfiguration ist IDE-spezifisch → nicht portabel

*In professionellen Projekten ersetzt IntelliJ **keine** Build-Tools – es integriert sie! IntelliJ hat eingebaute Unterstützung für Maven und Gradle und delegiert den eigentlichen Build an diese.*

Warum Build-Tools?

Probleme ohne Build-Tool:

- Viele Klassen → komplexe `javac`-Aufrufe
- Externe Bibliotheken müssen manuell heruntergeladen und verwaltet werden
- Kein einheitlicher Prozess → „Works on my machine“
- Testen, Kompilieren, Paketieren muss manuell koordiniert werden

Build-Tools lösen diese Probleme:

- **Automatisierung:** Kompilieren, Testen, Paketieren in einem Befehl
- **Dependency Management:** Bibliotheken werden automatisch heruntergeladen
- **Standardisierung:** Einheitliche Projektstruktur und Prozesse
- **Reproduzierbarkeit:** Gleiche Ergebnisse auf jedem Rechner

Apache Maven – Einführung

Apache Maven ist das meistverbreitete Build-Tool im Java-Ökosystem.

- **Convention over Configuration:** Standardisierte Projektstruktur
- **Dependency Management:** Bibliotheken aus dem [Maven Central Repository](#) werden automatisch geladen
- **Build Lifecycle:** Vordefinierter Ablauf von Build-Phasen
- **Plugins:** Erweiterbar durch hunderte von Plugins
- Konfiguration über eine einzige Datei: **pom.xml** (Project Object Model)

Maven – Standard-Projektstruktur

```
mein-projekt/  
├─ pom.xml  
│   └─ project configuration  
├─ src/  
│   ├── main/  
│   │   ├── java/  
│   │   │   ├── de/mbn/myapp/  
│   │   │   │   └─ MyApp.java ← production code  
│   │   └─ test/  
│   │       ├── java/  
│   │       │   ├── de/mbn/myapp/  
│   │       │   └─ MyAppTest.java ← test code  
└─
```

Maven erwartet diese Struktur – kein manuelles Konfigurieren nötig!

Maven – pom.xml (Aufbau)

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <!-- Project coordinates (unique identification) -->
  <groupId>de.mbn.myapp</groupId>      <!-- organization / domain -->
  <artifactId>mein-projekt</artifactId> <!-- project name -->
  <version>1.0.0</version>             <!-- version -->
  <packaging>jar</packaging>           <!-- output format -->

  <properties>
    <maven.compiler.source>21</maven.compiler.source>
    <maven.compiler.target>21</maven.compiler.target>
  </properties>

  <dependencies>
    <!-- External libraries go here -->
  </dependencies>

</project>
```

Maven – Dependency Management

*Externe Bibliotheken werden über **Koordinaten** (groupId, artifactId, version) eingebunden. Maven lädt sie automatisch aus dem **Maven Central Repository** herunter.*

```
<dependencies>
  <!-- JUnit 5 for unit tests -->
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.11.0</version>
    <scope>test</scope>    <!-- for tests only, not in production JAR -->
  </dependency>

  <!-- Example: logging library -->
  <dependency>
    <groupId>org.slf4j</groupId>
    <artifactId>slf4j-simple</artifactId>
    <version>2.0.13</version>
  </dependency>
</dependencies>
```

Dependency Scopes:

- `compile` (Standard): im Classpath beim Kompilieren und Ausführen
- `test`: nur beim Testen verfügbar
- `provided`: wird zur Laufzeit von der Umgebung bereitgestellt (z.B. Servlet-API)

Maven – Build Lifecycle

Maven kennt einen **Standard-Lifecycle** mit festen Phasen (Auswahl):

Phase	Beschreibung
<code>validate</code>	Prüft, ob das Projekt korrekt konfiguriert ist
<code>compile</code>	Kompiliert den Quellcode nach <code>target/classes/</code>
<code>test</code>	Führt Unit-Tests aus (schlägt fehl → Build schlägt fehl)
<code>package</code>	Paketiert den Code als JAR/WAR in <code>target/</code>
<code>verify</code>	Führt Integrationstests und Qualitätsprüfungen aus
<code>install</code>	Installiert das Paket in das lokale Maven-Repository (<code>~/ .m2/</code>)
<code>deploy</code>	Veröffentlicht das Paket in ein entferntes Repository

Jede Phase schließt alle vorherigen Phasen ein: `mvn package` führt auch `validate`, `compile` und `test` aus!

Maven – Wichtige Befehle

```
# Compile project
mvn compile

# Run tests
mvn test

# Generate JAR file (in target/)
mvn package

# Install JAR file into local repository
mvn install

# Delete build artifacts (target/ folder)
mvn clean

# Clean rebuild and generate JAR (most common workflow!)
mvn clean package

# Skip tests (e.g. with known failures – not recommended!)
mvn clean package -DskipTests
```

Die erzeugte JAR-Datei liegt nach `mvn package` im `target/`-Verzeichnis.

Maven – Zusammenfassung

Developer writes code

↓
mvn clean package

↓
validate → compile → test → package

| pom.xml is read

| Dependencies are loaded (Maven Central)
| Source code is compiled

| Unit tests are executed

| JAR file is generated

↓
target/mein-projekt-1.0.0.jar

↓
java -jar target/mein-projekt-1.0.0.jar

Innere Klassen

von inneren Klassen hin zu Lambda-Funktionen

Arten von inneren Klassen

- Statisch verschachtelte Klasse (Static Nested Class)
 - Geschachtelte **statische** Klasse innerhalb einer anderen Klasse mit Bezeichner (Klassenname)
 - können innerhalb und außerhalb (abhängig von der Sichtbarkeit) der Klasse verwendet werden
 - hat **keinen** Zugriff auf Instanzvariablen der äußeren Klasse (nur auf statische Mitglieder)
- Innere Element Klasse
 - Geschachtelte Klasse innerhalb einer anderen Klasse mit Bezeichner (Klassenname)
 - können innerhalb und außerhalb (abhängig von der Sichtbarkeit) der Klasse verwendet werden
 - nur im Kontext eines Objekts der äußeren Klasse
 - hat Zugriff auf **alle** Mitglieder der äußeren Klasse (auch `private`)
- Innere lokale Klasse
 - Geschachtelte Klasse innerhalb einer Methode mit Bezeichner (Klassenname)
 - können nur innerhalb der Methode (Scope) genutzt werden
 - hat Zugriff auf **alle** Mitglieder der äußeren Klasse (auch `private`)
- Innere anonyme Klasse
 - geschachtelte Klasse innerhalb einer anderen Klasse / Methode **ohne** Bezeichner
 - werden direkt einer Referenz zugewiesen oder als Methodenargument übergeben
 - basieren immer auf einer Klasse (erweitern) oder einem Interface (implementieren)
 - hat Zugriff auf **alle** Mitglieder der äußeren Klasse (auch `private`)

Statisch verschachtelte Klasse (Static Nested Class)

```
package main.inner.toplevelclass;

public class OuterClass {

    // Static nested class - kein Zugriff auf Instanzvariablen der äußeren Klasse
    public static class InnerTopLevelClass{
        void print(String printText){
            System.out.println(this.getClass().getName() + " " + printText);
        }
    }

    private static void printFromInnerTopLevelClass(String printText) {
        OuterClass.InnerTopLevelClass myInnerTopLevelClass = new OuterClass.InnerTopLevelClass();
        myInnerTopLevelClass.print(printText);
    }

    public static void main(String[] args) {
        OuterClass myClass = new OuterClass();

        System.out.println("OuterClass: " + myClass.getClass().getName());
        OuterClass.printFromInnerTopLevelClass("Static Nested Class: HelloWorld");
    }
}
```

Anwendungsbeispiel: Static Nested Class

Car.Statistics – Klassenweite Statistik, die kein konkretes Auto-Objekt braucht

```
public class Car {
    private static int carCount = 0;
    private String brand;
    private int kmh = 0;

    public Car(String brand) {
        this.brand = brand;
        Car.carCount++;
    }

    // Static Nested Class: Zugriff auf statische Mitglieder (carCount),
    // aber KEIN Zugriff auf Instanzvariablen (brand, kmh)
    public static class Statistics {
        public void printStats() {
            System.out.println("Erstellte Autos: " + Car.carCount);
        }
    }

    public static void main(String[] args) {
        new Car("BMW");
        new Car("VW");
        new Car("Audi");

        Car.Statistics stats = new Car.Statistics();
        stats.printStats();    // Erstellte Autos: 3
    }
}
```

Innere Element-Klasse

```
package main.inner.elementclass;

public class OuterClass {

    // Element class defined inside another class
    public class InnerElementClass {
        void print(String printText){
            System.out.println(this.getClass().getName() + " " + printText);
        }
    }

    void printFromInnerElementClass(String printText){
        // Innerhalb der äußeren Klasse: new InnerElementClass() reicht aus
        // Von außen wäre die Syntax: outerObj.new InnerElementClass()
        InnerElementClass myInnerElementClass = new InnerElementClass();

        myInnerElementClass.print(printText);
    }

    public static void main(String[] args) {
        OuterClass myClass = new OuterClass();

        System.out.println("OuterClass: " + myClass.getClass().getName());
        myClass.printFromInnerElementClass("Inner Element Class: HelloWorld");
    }
}
```

Anwendungsbeispiel: Innere Element-Klasse

Car.Engine – Der Motor gehört zum Auto und greift direkt auf dessen Zustand zu

```
public class Car {
    private String brand;
    private int kmh = 0;

    public Car(String brand) { this.brand = brand; }

    // Innere Element-Klasse: Direkter Zugriff auf brand und kmh der äußeren Instanz
    public class Engine {
        public void accelerate(int amount) {
            kmh += amount; // greift direkt auf kmh von Car zu
            System.out.println(brand + " beschleunigt auf " + kmh + " km/h");
        }
        public void brake(int amount) {
            kmh = Math.max(0, kmh - amount);
            System.out.println(brand + " bremst auf " + kmh + " km/h");
        }
    }

    public static void main(String[] args) {
        Car myCar = new Car("BMW");
        Car.Engine engine = myCar.new Engine(); // Von außen: outerObj.new InnerClass()

        engine.accelerate(50); // BMW beschleunigt auf 50 km/h
        engine.accelerate(30); // BMW beschleunigt auf 80 km/h
        engine.brake(20);      // BMW bremst auf 60 km/h
    }
}
```


Innere lokale Klasse

```
package main.inner.local;

public class OuterClass {

    void printFromLocalInnerClass(String printText){
        // class defined inside a method (scope)
        class LocalInnerClass{
            void print(String printText){
                System.out.println(this.getClass().getName() + " " + printText);
            }
        }

        LocalInnerClass myLocalInnerClass = new LocalInnerClass();

        myLocalInnerClass.print(printText);
    }

    public static void main(String[] args) {
        OuterClass myClass = new OuterClass();

        System.out.println("OuterClass: " + myClass.getClass().getName());
        myClass.printFromLocalInnerClass("local inner Class: HelloWorld");
    }
}
```

Anwendungsbeispiel: Innere lokale Klasse

StudentFormatter – Formatierungslogik, die nur innerhalb einer Methode gebraucht wird

```
import java.util.List;

public class StudentRegistry {
    private List<Student> students;

    public StudentRegistry(List<Student> students) { this.students = students; }

    public void printStudentList(String format) {
        // Lokale Klasse: nur innerhalb dieser Methode sichtbar und nutzbar
        class StudentFormatter {
            String format(Student s) {
                if (format.equals("short")) {
                    return s.getMatrikelNo() + ": " + s.getLastName();
                } else {
                    return s.getMatrikelNo() + ": " + s.getLastName() + ", " + s.getFirstName();
                }
            }
        }

        StudentFormatter formatter = new StudentFormatter();
        for (Student s : students) {
            System.out.println(formatter.format(s));
        }
    }

    public static void main(String[] args) {
        var registry = new StudentRegistry(List.of(
            new Student("Max", "Müller", 12345),
            new Student("Anna", "Schmidt", 67890)
        ));

        registry.printStudentList("short"); // 12345: Müller
        registry.printStudentList("long");  // 12345: Müller, Max
    }
}
```

Innere anonyme Klasse

```
package main.inner.anonym;

public class OuterClass {

    private static interface Printable{
        void print(String printText);
    }

    void printFromAnonymousInnerClass(String printText) {
        // defined without its own identifier (cannot be reused)
        // extends an existing class or implements an interface
        // 'this' refers to the anonymous class itself (not OuterClass!)
        OuterClass.Printable myAnonymousInnerClass = new OuterClass.Printable() {
            @Override
            public void print(String printText) {
                System.out.println(this.getClass().getName() + " " + printText);
            }
        };

        myAnonymousInnerClass.print(printText);
    }

    public static void main(String[] args) {
        OuterClass myClass = new OuterClass();

        System.out.println("OuterClass: " + myClass.getClass().getName());
        myClass.printFromAnonymousInnerClass("Inner anonymous Class: HelloWorld");
    }
}
```

Anwendungsbeispiel: Innere anonyme Klasse

Sortierung von `Student`-Objekten mit einem `Comparator` – einmalig, kein eigener Klassenname nötig

```
import java.util.*;

public class StudentSorter {
    public static void main(String[] args) {
        List<Student> students = new ArrayList<>();
        students.add(new Student("Max", "Müller", 12345));
        students.add(new Student("Anna", "Schmidt", 67890));
        students.add(new Student("Tom", "Bauer", 11111));

        // Anonyme Klasse implementiert Comparator<Student> (kein Bezeichner, einmalige Nutzung)
        // 'this' bezieht sich hier auf die anonyme Klasse, nicht auf StudentSorter!
        Collections.sort(students, new Comparator<Student>() {
            @Override
            public int compare(Student s1, Student s2) {
                return Integer.compare(s1.getMatrikelNo(), s2.getMatrikelNo());
            }
        });

        for (Student s : students) {
            System.out.println(s.getMatrikelNo() + ": " + s.getLastName());
        }
        // 11111: Bauer
        // 12345: Müller
        // 67890: Schmidt
    }
}
```

Lambda Funktionen (anonyme Funktionen)

- seit Java 8
- reine Funktionen ohne eigene Klasse
- Definition: () -> {}
- implementieren ein funktionales Interface (Interface mit **genau einer abstrakten** Methode – SAM: Single Abstract Method)
 - default- und static-Methoden im Interface sind erlaubt
 - ersetzen (unter dieser Voraussetzung) anonyme Klassen
- haben Zugriff auf den umliegenden Kontext (finale / effektiv finale Variablen)
 - in diesem Zusammenhang auch als "Closure" bezeichnet
- verkürzte Schreibweise durch Herleitung der Informationen aus Interface-Definition
- werden an eine Referenz übergeben (direkt oder indirekt)

```
Interface1 lambda1 = parameter -> statement;  
Interface2 lambda2 = (parameter1, parameter2) -> statement;  
Interface3 lambda3 = () -> {  
    statement1;  
    statement2;  
    statement3;  
}
```

Lambda Funktion

```
package main.lambda;

public class OuterClass {

    private static interface Printable{
        void print(String printText);
    }

    void printFromLambdaFunction(String printText) {
        // Lambda functions are "pure functions" without a class
        // always use a functional interface (SAM - Single Abstract Method)
        // for implementation
        // 'this' refers to OuterClass (not the lambda!) - key difference to anonymous classes
        OuterClass.Printable myLambdaPrintFunction = (lambdaPrintText) -> {
            System.out.println(this.getClass().getName() + " " + lambdaPrintText);
        };

        myLambdaPrintFunction.print(printText);
    }

    public static void main(String[] args) {
        OuterClass myClass = new OuterClass();

        System.out.println("OuterClass: " + myClass.getClass().getName());
        myClass.printFromLambdaFunction("Lambda Function: HelloWorld");
    }
}
```

Anwendungsbeispiel: Lambda

Dieselbe Student-Sortierung – von der anonymen Klasse zum Lambda in drei Schritten

```
import java.util.*;

public class StudentSorter {
    public static void main(String[] args) {
        List<Student> students = new ArrayList<>();
        students.add(new Student("Max", "Müller", 12345));
        students.add(new Student("Anna", "Schmidt", 67890));
        students.add(new Student("Tom", "Bauer", 11111));

        // Schritt 1: Anonyme Klasse (wie zuvor)
        Collections.sort(students, new Comparator<Student>() {
            @Override
            public int compare(Student s1, Student s2) {
                return Integer.compare(s1.getMatrikelNo(), s2.getMatrikelNo());
            }
        });

        // Schritt 2: Lambda – Comparator<Student> hat genau eine abstrakte Methode (SAM)
        // 'this' bezieht sich hier auf StudentSorter (nicht auf ein Lambda-Objekt!)
        Collections.sort(students,
            (s1, s2) -> Integer.compare(s1.getMatrikelNo(), s2.getMatrikelNo()));

        // Schritt 3: Noch kürzer mit Methoden-Referenz
        students.sort(Comparator.comparingInt(Student::getMatrikelNo));

        for (Student s : students) {
            System.out.println(s.getMatrikelNo() + ": " + s.getLastName());
        }
    }
}
```

Innere Klassen im JDK – Übersicht

Wo begegnen uns die vier Typen im Java Standard-Quellcode?

Typ	JDK-Beispiel	Package
Static Nested Class	<code>Map.Entry<K,V></code>	<code>java.util</code>
Static Nested Class	<code>AbstractMap.SimpleEntry</code>	<code>java.util</code>
Static Nested Class	<code>Character.UnicodeBlock</code>	<code>java.lang</code>
Innere Element-Klasse	<code>ArrayList.Itr</code>	<code>java.util</code>
Innere Element-Klasse	<code>LinkedList.ListItr</code>	<code>java.util</code>
Innere lokale Klasse	<i>(kaum vorhanden – zu wenig wartbar)</i>	–
Anonyme Klasse	<i>(vor Java 8 – heute meist Lambdas)</i>	–

JDK: Static Nested Class – Map.Entry

java.util.Map.Entry<K, V> – gehört konzeptuell zu Map, braucht aber kein Map-Objekt

```
// java.util.Map (vereinfacht)
public interface Map<K, V> {

    // static ist bei Interface-Membnern implizit
    // Entry gehört zur Map, ist aber unabhängig von einer konkreten Map-Instanz
    interface Entry<K, V> {
        K getKey();
        V getValue();
        V setValue(V value);
    }

    Set<Map.Entry<K, V>> entrySet();
}

// Verwendung: Iteration über alle Einträge einer Map
Map<String, Integer> scores = new HashMap<>();
scores.put("Alice", 95);
scores.put("Bob", 82);

for (Map.Entry<String, Integer> entry : scores.entrySet()) {
    System.out.println(entry.getKey() + " → " + entry.getValue());
}
```

Weitere Beispiele: AbstractMap.SimpleEntry, HashMap.Node<K, V>, Character.UnicodeBlock

JDK: Innere Element-Klasse – ArrayList.Itr

java.util.ArrayList – der Iterator greift direkt auf das interne Array zu

```
// java.util.ArrayList (stark vereinfacht)
public class ArrayList<E> {

    private Object[] elementData; // das interne Array
    private int size;
    int modCount;
    // zählt strukturelle Änderungen

    // Innere Element-Klasse: kein Getter nötig – direkter Zugriff auf Felder von ArrayList
    private class Itr implements Iterator<E> {
        int cursor = 0;
        int lastRet = -1;
        int expectedModCount = modCount;

        public boolean hasNext() {
            return cursor != size; // direkter Zugriff auf ArrayList.size
        }

        public E next() {
            checkForComodification();
            return (E) elementData[cursor++]; // direkter Zugriff auf ArrayList.elementData
        }
    }

    public Iterator<E> iterator() {
        return new Itr();
    }
}
```

Weitere Beispiele: *LinkedList.ListItr*,
HashMap.EntrySet, *TreeMap.EntrySet*

JDK: Anonyme Klasse – Vor Java 8

*java.lang.Runnable und java.util.Comparator –
klassische Einsatzgebiete vor Java 8*

```
// Vor Java 8: Thread mit anonymer Klasse (java.lang.Runnable)
Thread t = new Thread(new Runnable() {
    @Override
    public void run() {
        System.out.println("Läuft in einem eigenen Thread");
    }
});
t.start();

// Ab Java 8: Lambda – Runnable ist ein funktionales Interface (SAM)
new Thread(() -> System.out.println("Läuft in einem eigenen Thread")).start();
```

```
// Vor Java 8: Sortierung mit anonymer Comparator (java.util.Comparator)
Collections.sort(students, new Comparator<Student>() {
    @Override
    public int compare(Student s1, Student s2) {
        return s1.getLastName().compareTo(s2.getLastName());
    }
});

// Ab Java 8: Lambda
students.sort((s1, s2) -> s1.getLastName().compareTo(s2.getLastName()));
```

*Seit Java 8 wurden anonyme Klassen im JDK weitgehend durch
Lambdas ersetzt – lokale Klassen waren dort nie nennenswert
verbreitet.*

Streams

Streams (`java.io`) VS Streams (`java.util.stream`)

Stream - Grundkonzept

- übertragen von Elementen (Daten / Objekten) zwischen einer Quelle und einem Ziel

Stream (`java.util.stream`)

- einfache (besser lesbare) Modifikation von Objekt(-Strömen)
 - Ersatz (funktionales Paradigma) für sequentielle Abarbeitung wie z.B. Schleifen
- Erweiterung des Collection Framework
- Quelle von Interface Collection, Ziel von Interface Collection, einfache Datentypen, etc (abhängig von Funktion)

Stream (`java.io`)

- übertragen von Daten zu bzw. von externen Ressourcen
- I/O --> Input / Output
- Beispiele
 - lesen aus Dateien
 - Ausgabe auf der Konsole
 - senden von Daten über das Netzwerk

Konzept

- Aufbau
 - Erzeugende-Operationen
 - Zwischen-Operationen
 - End-Operationen
- Trägheit (Laziness)
- Nicht Inteferenz

Zwischen-Operationen

- [", ", ""].filter(") -->

Entwurfsmuster (Design Patterns)

Entwurfsmuster (Design Patterns) in der Software-Entwicklung

- Lösungsschablonen für wiederkehrende Probleme im Software-Entwurf
- zur Verbesserung der Softwarearchitektur, Strukturierung des Code und bessere Lesbarkeit
- stellen keine starren Regeln, sondern flexible Richtlinien, die an die jeweilige Situation angepasst werden können
- geprägt durch die **Gang of Four (GoF)** – Buch: *Design Patterns* (Gamma et al., 1994)

Übersicht: Kategorien von Entwurfsmustern

Kategorie	Zweck	Beispiele
Erzeugungs-Muster (Creational)	Objekterzeugung flexibel gestalten	Singleton, Factory Method, Builder
Struktur-Muster (Structural)	Beziehungen zwischen Klassen/Objekten strukturieren	Facade, Adapter, Composite
Verhaltens-Muster (Behavioral)	Kommunikation und Verantwortlichkeiten zwischen Objekten	Strategy, Observer, Command

Erzeugungsmuster (Creational Patterns)

- steuern die **Erzeugung von Objekten**
- entkoppeln die Konstruktion eines Objekts von seiner konkreten Klasse
- ermöglichen flexible Auswahl der Implementierung zur Laufzeit
- Beispiele: **Singleton**, **Factory Method**, **Builder**

Creational Pattern: Singleton

*Stellt sicher, dass von einer Klasse **genau eine Instanz** existiert und bietet einen globalen Zugriffspunkt darauf.*

```
public class DatabaseConnection {  
    private static DatabaseConnection instance;  
  
    private DatabaseConnection() {  
        // private constructor prevents direct instantiation  
    }  
  
    public static DatabaseConnection getInstance() {  
        if (instance == null) {  
            instance = new DatabaseConnection();  
        }  
        return instance;  
    }  
  
    public void query(String sql) { /* ... */ }  
}  
  
// Usage:  
DatabaseConnection db = DatabaseConnection.getInstance();
```

Creational Pattern: Factory Method

*Definiert eine Schnittstelle zur Objekterzeugung, überlässt aber **Subklassen** die Entscheidung, welche Klasse instanziiert wird.*

```
public interface Notification {
    void send(String message);
}

public class EmailNotification implements Notification {
    public void send(String message) { System.out.println("Email: " + message); }
}

public class SmsNotification implements Notification {
    public void send(String message) { System.out.println("SMS: " + message); }
}

public class NotificationFactory {
    public static Notification create(String type) {
        return switch (type) {
            case "email" -> new EmailNotification();
            case "sms"   -> new SmsNotification();
            default      -> throw new IllegalArgumentException("Unknown type: " + type);
        };
    }
}

// Usage:
Notification n = NotificationFactory.create("email");
n.send("Welcome!");
```

Creational Pattern: Builder

*Trennt die **schrittweise Konstruktion** eines komplexen Objekts von seiner Repräsentation.*

```
public class Pizza {
    private final String size;
    private final boolean cheese;
    private final boolean pepperoni;

    private Pizza(Builder builder) {
        this.size = builder.size;
        this.cheese = builder.cheese;
        this.pepperoni = builder.pepperoni;
    }

    public static class Builder {
        private final String size;
        private boolean cheese = false;
        private boolean pepperoni = false;

        public Builder(String size) { this.size = size; }
        public Builder cheese() { this.cheese = true; return this; }
        public Builder pepperoni() { this.pepperoni = true; return this; }
        public Pizza build()
    { return new Pizza(this); }
    }
}

// Usage:
Pizza pizza = new Pizza.Builder("Large").cheese().pepperoni().build();
```


Struktur-Muster (Structural Patterns)

- beschreiben, wie Klassen und Objekte zu größeren Strukturen **zusammengesetzt** werden
- nutzen Abstraktion, um komplexe Strukturen flexibel und erweiterbar zu halten
- ermöglichen die Wiederverwendung von Klassen mit inkompatiblen Schnittstellen
- Beispiele: **Facade, Adapter, Composite**

Structural Pattern: Facade

Bietet eine **vereinfachte Schnittstelle** zu einem komplexen **Subsystem**.

```
// Complex subsystem
class CpuSubsystem { void start() { System.out.println("CPU started"); } }
class MemorySubsystem { void load() { System.out.println("Memory loaded"); } }
class DiskSubsystem { void read() { System.out.println("Disk read"); } }

// Facade: simple interface to the outside
public class ComputerFacade {
    private final CpuSubsystem cpu = new CpuSubsystem();
    private final MemorySubsystem memory = new MemorySubsystem();
    private final DiskSubsystem disk = new DiskSubsystem();

    public void startComputer() {
        cpu.start();
        memory.load();
        disk.read();
        System.out.println("Computer started!");
    }
}

// Usage:
ComputerFacade computer = new ComputerFacade();
computer.startComputer(); // subsystem details hidden
```

Structural Pattern: Adapter

*Passt die **Schnittstelle einer Klasse** an eine andere, vom Client erwartete Schnittstelle an.*

```
// Existing, incompatible class (e.g. third-party)
public class EuroSocket {
    public void plugIn220V() { System.out.println("220V power"); }
}

// Target interface expected by the client
public interface UsbCharger {
    void charge();
}

// Adapter: connects EuroSocket with UsbCharger
public class SocketAdapter implements UsbCharger {
    private final EuroSocket socket;

    public SocketAdapter(EuroSocket socket) { this.socket = socket; }

    @Override
    public void charge() {
        socket.plugIn220V(); // calls incompatible method internally
        System.out.println("-> Converted to USB");
    }
}

// Usage:
UsbCharger charger = new SocketAdapter(new EuroSocket());
charger.charge();
```

Verhaltens-Muster (Behavioral Patterns)

- beschreiben die **Kommunikation und Zusammenarbeit** zwischen Objekten zur Laufzeit
- legen fest, wie Verantwortlichkeiten zwischen Objekten verteilt werden
- erhöhen die Flexibilität bei der Ausführung von Algorithmen und der Reaktion auf Ereignisse
- Beispiele: **Strategy, Observer, Command**

Behavioral Pattern: Strategy

*Definiert eine Familie von Algorithmen, kapselt sie und macht sie **austauschbar** – ohne den nutzenden Code zu ändern.*

```
// Strategy-Interface
public interface SortStrategy {
    void sort(int[] data);
}

// Concrete strategies
public class BubbleSort implements SortStrategy {
    public void sort(int[] data) { System.out.println("BubbleSort executed"); }
}

public class QuickSort implements SortStrategy {
    public void sort(int[] data) { System.out.println("QuickSort executed"); }
}

// Context: uses a strategy
public class Sorter {
    private SortStrategy strategy;

    public Sorter(SortStrategy strategy) { this.strategy = strategy; }

    public void setStrategy(SortStrategy strategy) { this.strategy = strategy; }

    public void sort(int[] data) { strategy.sort(data); }
}

// Usage:
Sorter sorter = new Sorter(new BubbleSort());
sorter.sort(new int[]{3, 1, 2});
sorter.setStrategy(new QuickSort()); // switch strategy at runtime
sorter.sort(new int[]{3, 1, 2});
```

Behavioral Pattern: Observer

*Definiert eine **1-zu-n Abhängigkeit**: bei Zustandsänderung eines Objekts werden alle abhängigen Objekte automatisch benachrichtigt.*

```
import java.util.ArrayList;
import java.util.List;

public interface Observer {
    void update(String event);
}

public class EventSystem {
    private final List<Observer> observers = new ArrayList<>();

    public void subscribe(Observer o) { observers.add(o); }
    public void unsubscribe(Observer o) { observers.remove(o); }

    public void notifyObservers(String event) {
        for (Observer o : observers) {
            o.update(event);
        }
    }
}

// Usage:
EventSystem events = new EventSystem();
events.subscribe(e -> System.out.println("Logger: " + e));
events.subscribe(e -> System.out.println("UI-Update: " + e));

events.notifyObservers("File saved");
// Logger: File saved
// UI-Update: File saved
```

Optionals

java.util.Optional – ein Container-Objekt für potenziell nicht vorhandene Werte (seit Java 8)

Das Problem: null

- null bedeutet "kein Wert" – aber Java-Referenzen können immer null sein
- Zugriff auf ein null-Objekt führt zur gefürchteten NullPointerException
- Klassisches Muster – fehleranfällig und schwer lesbar:

```
public String getStudentCity(Student student) {  
    if (student != null) {  
        Address address = student.getAddress();  
        if (address != null) {  
            return address.getCity();  
        }  
    }  
    return "Unknown";  
}
```

- Probleme:
 - Entwickler vergessen null-Prüfungen
 - verschachtelte if-Blöcke reduzieren Lesbarkeit
 - null als Rückgabewert ist nicht selbsterklärend

Optional – Grundidee

- `Optional<T>` ist ein Container, der **entweder einen Wert enthält oder leer ist**
- macht das Fehlen eines Wertes **explizit** im Typ-System sichtbar
- zwingt den Aufrufer, den "kein Wert"-Fall zu behandeln
- befindet sich im Package `java.util`

```
// Without Optional: unclear whether null can be returned
public String findCity() { ... }

// With Optional: explicit that no value is possible
public Optional<String> findCity() { ... }
```

Erzeugen von Optional

- `Optional.of(value)` – erzeugt ein Optional mit dem gegebenen Wert (**nicht null!**)
- `Optional.ofNullable(value)` – erzeugt ein Optional; bei `null` → leeres Optional
- `Optional.empty()` – erzeugt ein leeres Optional

```
Optional<String> withValue = Optional.of("Berlin");  
Optional<String> nullable = Optional.ofNullable(getCity()); // getCity() may return null  
Optional<String> empty    = Optional.empty();  
  
Optional<String> error     = Optional.of(null);  
// --> NullPointerException!
```

Wichtige Methoden (1/2)

Prüfen & Wert holen:

- `isPresent()` – gibt `true` zurück, wenn ein Wert vorhanden ist
- `isEmpty()` – gibt `true` zurück, wenn kein Wert vorhanden ist (seit Java 11)
- `get()` – gibt den Wert zurück; wirft `NoSuchElementException` wenn leer → **mit Vorsicht verwenden!**

Sichere Alternativen zu `get()`:

- `orElse(T other)` – gibt den Wert zurück oder den Standardwert `other`
- `orElseGet(Supplier)` – gibt den Wert zurück oder berechnet einen Standardwert per Lambda
- `orElseThrow(Supplier)` – gibt den Wert zurück oder wirft eine eigene Exception

```
Optional<String> city = Optional.ofNullable(getCity());  
  
String c1 = city.orElse("Unknown");  
String c2 = city.orElseGet(() -> loadDefaultCity());  
String c3 = city.orElseThrow(() -> new IllegalStateException("No city!"));
```

Wichtige Methoden (2/2)

Aktionen ausführen:

- `ifPresent(Consumer)` – führt eine Aktion aus, wenn ein Wert vorhanden ist
- `ifPresentOrElse(Consumer, Runnable)` – Aktion bei Wert oder Alternativ-Aktion bei leer (seit Java 9)

Transformieren:

- `map(Function)` – transformiert den Wert, falls vorhanden (ähnlich wie bei Streams)
- `filter(Predicate)` – gibt das Optional zurück, wenn der Wert das Prädikat erfüllt, sonst leeres Optional
- `flatMap(Function)` – wie `map`, aber wenn die Funktion selbst ein Optional zurückgibt

```
Optional<String> city = Optional.ofNullable(student.getAddress())
    .map(Address::getCity)
    .filter(c -> !c.isBlank());

city.ifPresentOrElse(
    c -> System.out.println("City: " + c),
    () -> System.out.println("No city found")
);
```

Beispiel: Vorher / Nachher

Without Optional:

```
public String getStudentCity(Student student) {  
    if (student != null) {  
        Address address = student.getAddress();  
        if (address != null) {  
            String city = address.getCity();  
            if (city != null && !city.isBlank()) {  
                return city;  
            }  
        }  
    }  
    return "Unknown";  
}
```

With Optional:

```
public String getStudentCity(Student student) {  
    return Optional.ofNullable(student)  
        .map(Student::getAddress)  
        .map(Address::getCity)  
        .filter(city -> !city.isBlank())  
        .orElse("Unknown");  
}
```

Wann `Optional` verwenden?

Empfohlen:

- als **Rückgabety**p von Methoden, wenn kein Wert ein gültiges Ergebnis ist
 - `public Optional<Student> findById(int id) { ... }`

Nicht empfohlen:

- als **Methodenparameter** (nutze stattdessen Überladung oder Default-Werte)
- als **Attribut in Klassen** (`Optional` ist nicht serialisierbar)
- für **Collections** – eine leere Collection ist besser als `Optional<List<...>>`

```
// Good:
public Optional<String> findEmail(int userId) { ... }

// Avoid:
public void sendMail(Optional<String> email) { ... } // Not recommended
```

***Faustregel:** `Optional` macht das **Fehlen eines Rückgabewertes explizit** – nicht mehr und nicht weniger.*

Programming Principals

DRY, KISS, ... TODO... :-)

Principal Collection

- KISS
- DRY
- FIRST